IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 21 July 2025, accepted 11 September 2025, date of publication 22 September 2025,
date of current version 26 September 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3612407

RESEARCH ARTICLE

A Self-Imputing Deep Multitask Sequence Model for Traffic Disruption Detection in Extreme Conditions

MINA NOURI AND MOHAMMAD ILBEIGI
Stevens Institute of Technology, Hoboken, NJ 07030, USA
Corresponding author: Mohammad Ilbeigi (milbeigi@stevens.edu)

This work was supported by the National Science Foundation under Grant CMMI-2026795.

ABSTRACT Abnormal traffic patterns caused by extreme events can significantly disrupt traffic flow across large regions of urban road networks. Timely and reliable detection of such disruptions is crucial for effective traffic management. However, existing anomaly detection methods cannot simultaneously detect disruptions at both network and local levels, and they typically struggle with incomplete traffic data, which is a common issue during extreme events. To address these limitations, we propose a novel Self-Imputing Deep Multitask Sequence (SI-DMSeq) model for dual-level traffic monitoring and disruption detection. The model simultaneously performs two distinct tasks: 1) reconstructing sequences of traffic data, which enables anomaly detection through an autoencoder-based approach, and 2) making one-step-ahead predictions, which serves to impute missing data for the subsequent time step. To support robust training despite data gaps, we incorporate an iterative imputation strategy during the training phase. During inference, the reconstruction task is used to detect traffic disruptions at both network and local levels, while the prediction task provides reliable imputation of missing values under typical traffic conditions. We evaluate the SI-DMSeq model using historical traffic data from the Manhattan road network in New York City, focusing on disruptions caused by Hurricane Sandy in 2012 and the Christmas blizzard in 2010. The results demonstrate that the proposed method accurately identifies disruptions at both the network and local levels and efficiently imputes missing traffic data.

INDEX TERMS Anomaly detection, data imputation, multitask learning, urban traffic networks.

I. INTRODUCTION

In the face of extreme events, the importance of traffic monitoring and disruption detection within urban road networks cannot be overstated. These events, whether natural or human-made, often cause significant disruptions in normal traffic flow, which can lead to widespread congestion across the network. Timely and reliable detection of such disruptions enables informed traffic management to mitigate the adverse impacts of the event on urban traffic condition. Therefore, it is essential to develop a methodology that not only evaluates the overall traffic conditions at the network level but also pinpoints specific regions that experience the most severe effects. This dual-level approach enables transportation authorities to implement targeted interventions where they are most needed while simultaneously ensuring that the network as a whole remains functional and resilient.

Despite advancements in traffic monitoring, existing methods face several limitations. Many approaches either focus on detecting disruptions in specific road segments or attempt network-wide detection without pinpointing the most affected areas. Additionally, incomplete traffic data remains a significant challenge, as missing data can degrade the performance of detection models.

Motivated by these limitations, we propose a novel and comprehensive approach for detecting traffic disruptions during extreme events, at both network and local levels, while handling incomplete traffic data. Our method is designed to achieve three key objectives:

1. Identifying significant traffic disruptions resulting from extreme events at the network level.

The associate editor coordinating the review of this manuscript and approving it for publication was Yanli Xu.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

2. Pinpointing the regions most impacted by such events at a small zone level.
3. Imputing missing traffic data during the monitoring process.

More specifically, we introduce a Self-Imputing Deep Multitask Sequence (SI-DMSeq) model designed to perform two primary tasks: reconstruction and prediction. This model analyzes sequential traffic data, simultaneously reconstructs the input sequence and forecasts traffic data for the subsequent timestep. In our framework, these components are referred to as the 'Autoencoder' and the 'Future Predictor,' respectively. The model is trained using normal traffic data and incorporates an iterative imputation strategy to handle missing values within the training dataset. The trained Autoencoder captures normal traffic patterns but exhibits high reconstruction errors under abnormal traffic conditions. During inference, we leverage this property to detect large-scale disruptions by using a statistical control chart that identifies anomalous reconstruction loss values. Additionally, we propose an innovative scoring approach to identify the most disrupted regions within the network. The Future Predictor component ensures continuity in monitoring by imputing missing values for subsequent timesteps.

The key innovation of our approach lies in its dual-level disruption detection capability and the seamless integration of imputation and anomaly detection within a single multitask framework. Unlike existing methods that are limited to a single task or detection level, our model performs both tasks concurrently and enables dual-level detection. The integration of a statistical control chart for reconstruction-based anomaly detection, a scoring mechanism for localizing severe disruptions, and prediction-based imputation for handling missing values enables the monitoring system to operate effectively even in the presence of incomplete data. This cohesive design addresses major limitations in prior work and facilitates robust, dual-level monitoring of urban traffic conditions during extreme events.

The remainder of this paper is structured as follows: Section II provides a comprehensive review of related work on urban traffic disruption detection. Section III presents our methodology in detail. In Section IV, we validate our proposed method through real-world case studies and demonstrate its effectiveness. Finally, Section V summarizes our findings and discusses potential future research directions.

II. RELATED WORK

Traffic disruption detection has been the subject of numerous studies. These studies can be broadly categorized into three main groups:

The first group focuses on detecting disruptions in a specific road segment or a small number of roads. A subset of studies in this group (e.g., [1], [2], [3], [4]) employs traffic prediction techniques, where historical data is used to forecast short-term traffic conditions. Significant deviations from predicted values are then identified as disruptions. For example, Harrou et al. [3] introduced a traffic estimation methodology that combines the strengths of a piecewise switched linear traffic (PWSL) model and a Kalman filter (KF) model. They further proposed a K-nearest neighbors (KNN)-based technique to identify and flag significant deviations between the output of the PWSL-KF estimator and actual observations as abnormal traffic conditions. In another study, Evans et al. [4] proposed a context-based random forest model to predict traffic flow and raise alerts when observed values persistently fall outside prediction intervals. While effective for local disruptions, these methods face challenges when applied to large-scale networks. In such networks, the large number of road segments leads to a significant number of false alarms. Another subset of studies in the first group employs classification models to distinguish between normal and abnormal traffic states. Machine learning algorithms, such as Random Forest (e.g., [5], [6]), Support Vector Machines (e.g., [7], [8], [9]), and Neural Networks (e.g., [10], [11], [12], [13]), are commonly used for this purpose. However, the performance of these models significantly drops with imbalanced training data [14]. This challenge becomes particularly crucial in the context of extreme events, where the limited availability of event samples impedes the training of an effective model.

The second group of studies aims to expand traffic disruption detection methods to larger regions by including a greater number of road segments (e.g., [15], [16], [17], [18]). These studies typically employ decomposition-based approaches to capture the inherent spatiotemporal correlations in regular traffic and use them to detect disruptions. For example, Tisljaric et al. [16] utilized tensor decomposition techniques to analyze traffic patterns by transforming GPS-based traffic data into a Speed Transition Matrix (STM). They then identified disruptions by assessing the significance of the Kullback–Leibler divergence between the observed and average traffic patterns. Li et al. [17] proposed a coupled scalable Bayesian robust tensor factorization (coupled SBRTF) model to identify non-recurrent abnormal traffic patterns. Their model separates regular traffic from non-recurrent events by decomposing traffic data into a low-rank component (representing regular traffic) and a sparse outlier component (representing non-recurrent events). While the methods presented in these studies do not suffer from the false alarm issue faced by the studies in the first group, they are limited to offline analysis, as they retroactively detect abnormalities by analyzing all the data collected over a specific period. Therefore, they are not applicable for near real-time disruption detection.

The third group of studies addresses disruption detection on a network-wide scale (e.g., [19], [20], [21]). These studies aim to identify large-scale disruptions resulting from extreme events. For example, Donovan and Work [19] analyzed taxi GPS data to create probability distributions of typical travel paces between various regions of New York City. They subsequently used these distributions to detect major disruptions caused by extreme events. A more recent study by Hu et al. [21] proposed an autoencoder-based method to detect extreme events in origin-destination

mobility data, represented as time-dependent graphs. Their method employs embedding techniques to capture typical traffic patterns and utilizes an autoencoder framework to identify anomalous graphs associated with extreme events. While these network-wide approaches excel at identifying large-scale events, they cannot pinpoint specific regions that are most affected by such events.

In addition to the aforementioned limitations in the existing methods for simultaneously detecting network-level and local-level traffic disruptions, incomplete traffic data poses a critical challenge in devising traffic monitoring methods as well. Despite advancements in traffic data collection methods, data gathered from large-scale road networks is often incomplete. This incompleteness presents challenges for effective traffic monitoring and disruption detection. Imputing missing data without considering the underlying spatiotemporal patterns may result in the insertion of erroneous and inaccurate data points. These inaccuracies can subsequently degrade the performance of a disruption detection model. To address this issue, Hu and Work [22] introduced a method based on the robust tensor decomposition approach to identify traffic disruptions in partially observed data, while imputing missing data under typical traffic conditions. However, their method detects disruptions retroactively and is not suitable for online monitoring of road networks. This limitation prohibits the application of real-time or near real-time traffic monitoring.

Furthermore, traffic assignment models have been explored in the literature as a means for estimating network traffic conditions and predicting missing data points. These models rely on equilibrium-based or dynamic assignment principles to infer link-wise traffic flow from demand and network constraints (e.g., [23], [24], [25], [26]). While such models can provide reasonable estimates of typical traffic patterns, they are often limited by their reliance on static or semi-static assumptions and may fail to capture abrupt disruptions effectively. Additionally, their computational complexity makes them less practical for real-time or near-real-time applications.

This paper aims to overcome these challenges by introducing a near real-time disruption detection framework that simultaneously detects both network-wide and localized traffic anomalies while handling missing data effectively.

III. METHODOLOGY

In this section, we begin by providing a detailed explanation of the SI-DMSeq model. Following that, we present our data imputation approach. Subsequently, we introduce the dynamic loss function used for training the SI-DMSeq model. Lastly, we delve into traffic disruption detection at both the network and zone levels.

A. THE SI-DMSEQ MODEL ARCHITECTURE

Our proposed model follows an encoder-decoder architecture. It is designed for multitask sequence-to-sequence learning [27] and employs Long Short-Term Memory (LSTM) networks [28] for handling sequential traffic data. This model consists of a deep encoder LSTM and two separate deep decoder LSTMs, both leveraging a shared encoder. The encoder LSTM is tasked with analyzing sequential traffic data to capture its contextual information and extract a meaningful representation of the input sequence. Subsequently, the first decoder reconstructs the input sequence based on this learned representation. Concurrently, the second decoder forecasts traffic data for the next timestep using the encoded information obtained from the shared encoder.

The combination of both reconstruction and prediction tasks helps address the limitations inherent in individual models. A standalone Autoencoder may learn trivial representations that merely memorize the input sequence. Additionally, a standalone Future Predictor might demonstrate a bias toward recent data points and overlook broader patterns. However, the simultaneous learning of both reconstruction and prediction tasks serves as a regularization mechanism. The reconstruction task necessitates capturing detailed features of the input sequence, while the prediction task demands the ability to generalize and extrapolate from these learned features. By jointly optimizing for both tasks, the combined model is encouraged to learn a more informative and generalized representation of the input sequence. This representation retains information about the entire sequence and captures meaningful features beneficial for both reconstruction and prediction tasks.

Fig. 1 illustrates the architecture of the proposed SI-DMSeq model. A detailed explanation of each component is provided in the remainder of this section.

1) SHARED ENCODER

The shared encoder consists of L stacked LSTM layers. Each layer operates on the input data sequentially to capture its contextual information and generate an encoded representation of the input sequence. Specifically, given an input sequence $X = \{x_1, x_2, \cdots, x_T\}$, where T represents the sequence length and $x_t \in R^Z$ is a Z-dimensional vector of traffic values at timestep t, the shared encoder processes input vectors sequentially and maps them into a series of hidden state vectors, denoted by $\{h_1^L, h_2^L, \cdots, h_T^L\}$, where $h_t^L \in \mathbb{R}^h$ represents the upper-layer hidden state generated at timestep t, with h denoting the hidden size. These vectors contain the encoded information of the sequential data and capture the dependencies and patterns within the input sequence. The final hidden and cell states of the encoder layers (i.e., $h_T^l, c_T^l, l \in \{1, \cdots, L\}$) are used as the initial states for the two downstream decoders.

2) RECONSTRUCTION DECODER

The reconstruction decoder is tasked with recreating the input sequence using the encoded information obtained from the shared encoder. This decoder comprises L stacked LSTM layers followed by a Dense layer, which collectively work towards reconstructing the original input. It also leverages an Attention mechanism [29] to dynamically focus on

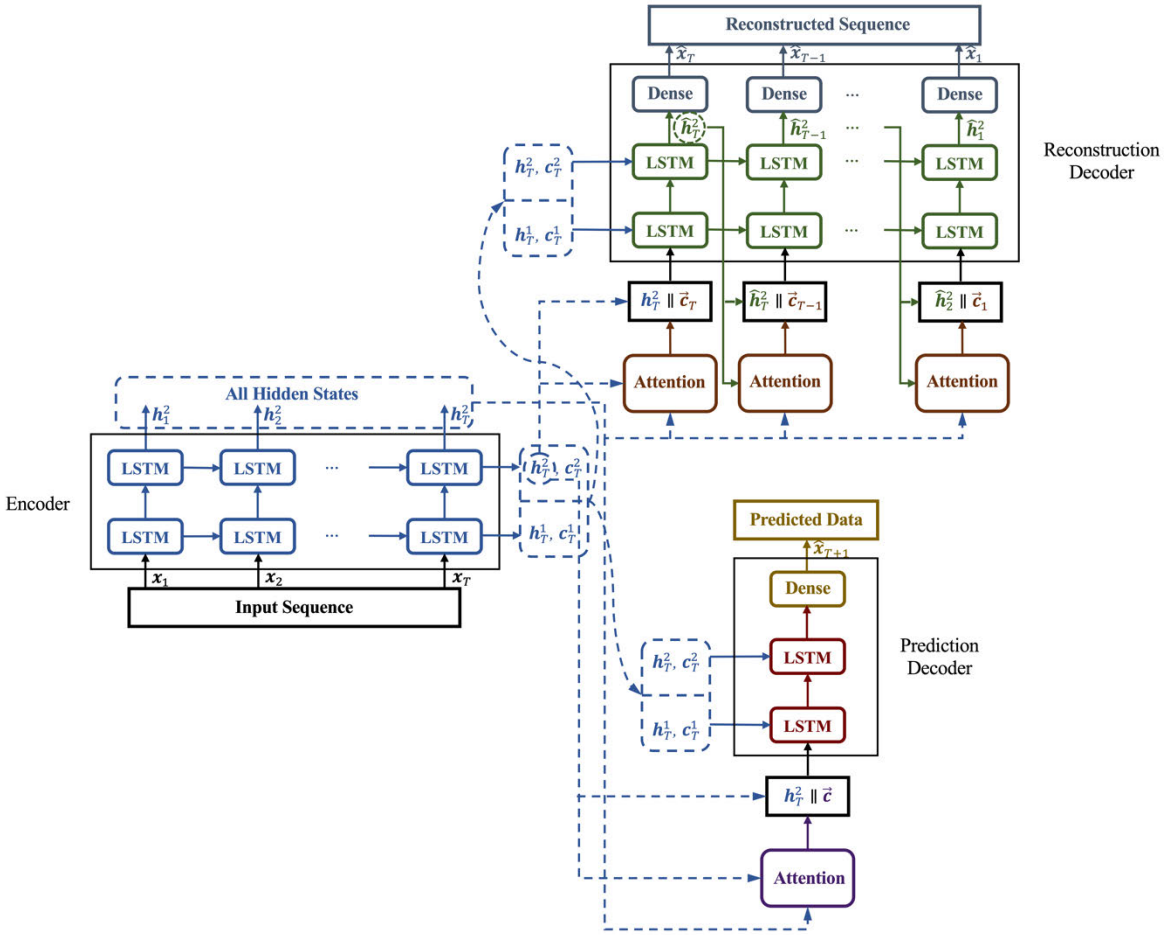


FIGURE 1. The architecture of the SI-DMSeq model.

relevant parts of the encoded sequence during the reconstruction process. Specifically, at each decoding step d, the attention mechanism computes a context vector $\vec{c}_d$ that represents the important parts of the input sequence for generating the current output. Section III-B. describes the steps involved in computing the context vector. This context vector is concatenated with the previous decoder hidden state and passed through the decoder layers to produce the output. The objective of the reconstruction decoder is to generate an output sequence of length T that ideally matches the input sequence in reverse order.

3) PREDICTION DECODER

The prediction decoder is designed to forecast traffic data for the subsequent timestep using the encoded information obtained from the shared encoder. Similar to the reconstruction decoder, it consists of L LSTM layers followed by a Dense layer and employs an Attention mechanism for effective future prediction. Upon encoding the input sequence, the Attention mechanism computes a context vector $\vec{c}$ that highlights the most relevant information within the input sequence for predicting subsequent traffic data. This context vector is then concatenated with the last hidden state of the encoder (i.e., h_T^L) before being used as input for the prediction decoder. The output of this decoder is a vector containing predicted traffic values for the next timestep in the sequence (i.e., x_{T+1}).

B. ATTENTION MECHANISM

The attention mechanism allows the model to dynamically focus on different parts of the input sequence during the decoding process. This mechanism employs a scoring function to compute alignment for every timestep in the input sequence relative to the ongoing decoding step. These scores highlight parts of the input sequence that are most relevant for generating the current output.

In the proposed SI-DMSeq model, we adopt a widely-used scoring function that utilizes a feed-forward neural network with a single hidden layer and a tanh activation function. At each decoding step d, the alignment scores are computed by considering the encoder's upper-layer hidden states across all time steps (i.e. $h_t^L, t \in \{1, \cdots, T\}$), along with the decoder's upper-layer hidden state from the previous decoding step (i.e. $\hat{h}_{d-1}^L$):

$$a\left(h_t^L, \hat{h}_{d-1}^L\right) = v_a^T \tanh\left(W_a\left[h_t^L; \hat{h}_{d-1}^L\right]\right) \quad (1)$$

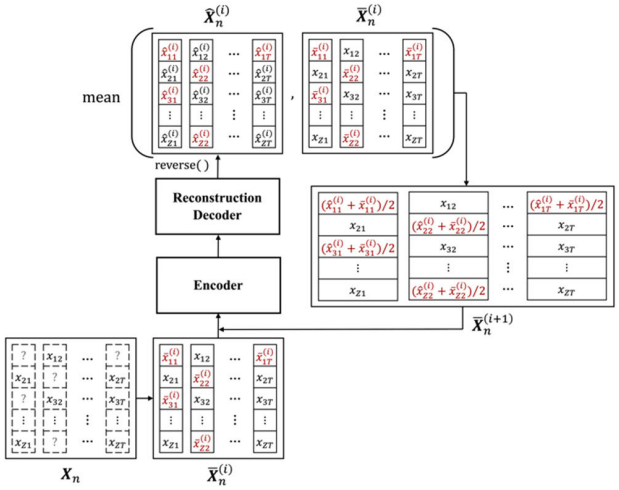


FIGURE 2. Data imputation process during model training.

here, $W_a \in \mathbb{R}^{h\times 2h}$ and $v_a \in \mathbb{R}^h$ are learnable parameters and $\left[h_t^L; \hat{h}_{d-1}^L\right]$ represents the concatenation of the encoder and decoder hidden states.

After calculating the scores, a Softmax function is used to obtain the corresponding attention weights. This ensures that the weights add up to one and represent a probability distribution.

$$\alpha_{d,t} = \text{softmax}\left(a\left(h_t^L, \hat{h}_{d-1}^L\right)\right) \tag{2}$$

Then, the context vector $\vec{c}_d$ is computed as the weighted sum of the encoder hidden states using the attention weights.

$$\vec{c}_d = \sum_{t=1}^{T} \alpha_{d,t} h_t^L, \tag{3}$$

where, T is the length of the input sequence. This context vector captures the relevant information from the encoder hidden states for the current decoding step.

Finally, the context vector $\vec{c}_d$ is concatenated with the previous decoder hidden state $\hat{h}_{d-1}^L$ before being used as input for the current decoder LSTM cell.

C. MISSING DATA IMPUTATION

Our proposed SI-DMSeq model integrates an iterative data imputation strategy directly within the training process. Rather than treating imputation as a separate step, our model simultaneously handles missing values while learning the underlying traffic patterns. Specifically, we adopt the method introduced by Pan et al. [30], which leverages the Autoencoder component of the model to iteratively impute missing data. Initially, missing values are imputed with the average of observed values from corresponding timesteps. As training progresses, these imputed values are progressively refined with the mean of the input data and the reconstructed data from each forward propagation step. This iterative imputation process allows the model to infer missing information based on the underlying distribution of the observed data. Upon completion of the training phase, both the imputed values and the learned model parameters are obtained.

The mathematical equations for this approach are as follows:

$$\begin{aligned}
\bar{X}_n^{(1)} &=\sim M_{x,n} \circ X_n + M_{x,n} \circ \text{mean}_1\left(X_n\right) \\
\hat{X}_n^{(i)}, \hat{y}_n^{(i)} &= \text{model}\left(\bar{X}_n^{(i)}\right), i = 1, 2, \cdots \\
\bar{X}_n^{(i+1)} &=\sim M_{x,n} \circ \bar{X}_n^{(i)} + M_{x,n} \circ \left(\hat{X}_n^{(i)} + \bar{X}_n^{(i)}\right) \Big/ 2
\end{aligned} \tag{4}$$

where the matrix $X_n \in \mathbb{R}^{Z\times T}$ denotes the nth sequence sample in the training dataset, with Z and T representing the number of traffic values and the number of timesteps, respectively. The binary matrix $M_{x,n} \in \mathbb{R}^{Z\times T}$ indicates the positions of missing values within X_n. The symbol $\circ$ denotes elementwise multiplication, and the $\sim$ operator performs a bitwise negation, changing each 0 to 1 and each 1 to 0. The function $\text{mean}_1(\cdot)$ computes the average of all non-missing values along the first dimension of the incomplete sample X_n, and $\text{model}(\cdot)$ denotes the forward propagation step within the iterative process. At the ith epoch, $\hat{X}_n^{(i)}$ represents the reconstructed input sequence, and $\hat{y}_n^{(i)}$ denotes the predicted traffic vector for the subsequent timestep. $\bar{X}_n^{(i+1)}$ is the imputed sequence sample, where missing values are substituted with the mean of the input sequence and its reconstruction obtained from the forward propagation. This imputed sequence serves as the input for the model in the $(i+1)$th epoch. Fig. 2 depicts the data imputation process during training.

To ensure the continuity of analysis during the inference phase, the model employs its prediction component (i.e., Future Predictor) to fill in missing values for the subsequent time step.

D. DYNAMIC LOSS FUNCTION AND MODEL TRAINING

Our proposed model utilizes a dynamic loss function to effectively impute missing data while prioritizing reconstruction and prediction accuracy. This function dynamically adapts its focus throughout the training process. In the early epochs, it emphasizes reconstructing and predicting non-missing values. This ensures that the model captures the underlying patterns in the observed data. As training progresses, the loss function gradually shifts its focus towards imputing missing values and eventually achieves a balance between the reconstruction and prediction of non-missing values and the imputation of missing ones. This adaptive approach enables the model to effectively infer missing information based on the observed context within the training data, thereby enhancing its performance in sequence reconstruction and prediction tasks. The dynamic loss function is defined as follows:

$$\begin{aligned}
\mathcal{L} = \sum_{n=1}^{N} \Bigg(&\left\| \left(\alpha^{(i)}\left(\sim M_{x,n}\right) + \beta^{(i)} M_{x,n}\right) \circ \left(\hat{X}_n^{(i)} - \bar{X}_n^{(i)}\right) \right\|_F^2 \\
&+ \left\| \alpha^{(i)}\left(\sim m_{y,n}\right) \circ \left(\hat{y}_n^{(i)} - y_n\right) \right\|^2 \Bigg), \\
\alpha^{(i)} = 2 - \left(i/epochs\right)^2 &; \beta^{(i)} = \left(i/epochs\right)^2
\end{aligned} \tag{5}$$

Algorithm 1

Input: missing sequence data $\mathcal{X} \in \mathbb{R}^{N\times T\times Z}$ and its corresponding binary mask tensor $M_x \in \mathbb{R}^{N\times T\times Z}$, missing target data $Y \in \mathbb{R}^{N\times Z}$ and its corresponding binary mask matrix $\mathcal{M}_y \in \mathbb{R}^{N\times Z}$, maximum number of *epochs*, batch size B

$\bar{\mathcal{X}}^{(1)} = \sim \mathcal{M}_x \circ \mathcal{X} + \mathcal{M}_x \circ mean_1(\mathcal{X})$;
$N = len(\mathcal{X})$;
for $i = 1$ **to** *epochs* **do**
  $\alpha^{(i)} = 2 - \left(i/epochs\right)^2$;
  $\beta^{(i)} = \left(i/epochs\right)^2$
  get a randomly shuffled list of unique integers
  $P = [p_j]_{j=1}^{N}$ ranging from 1 to N;
  $k = 0$
  repeat
    draw B integers from P, denoted as inds $= [p_j]_{j=1+k*B}^{\min((k+1)*B,N)}$
    get a batch of samples from dataset using the drawn integers inds, denoted as $\left(\bar{\mathcal{X}}_b^{(i)}, \mathcal{M}_{xb}, Y_b, M_{yb}\right)$;
    $\hat{\mathcal{X}}_b^{(i)}, \hat{Y}_b^{(i)} = \text{model}\left(\bar{\mathcal{X}}_b^{(i)}\right)$;
    $\text{loss} = \left\| \left(\alpha^{(i)}\left(\sim \mathcal{M}_{x,b}\right) + \beta^{(i)}\mathcal{M}_{x,b}\right) \circ \left(\hat{\mathcal{X}}_b^{(i)} - \bar{\mathcal{X}}_b^{(i)}\right) \right\|_F^2 + \left\| \alpha^{(i)}\left(\sim M_{y,b}\right) \circ \left(\hat{Y}_b^{(i)} - Y_b\right) \right\|_F^2$;
    backpropagate loss and update model parameters;
    $\bar{\mathcal{X}}_b^{(i+1)} = \sim M_{x,b} \circ \bar{\mathcal{X}}_b^{(i)} + \mathcal{M}_{x,b} \circ \left(\hat{\mathcal{X}}_b^{(i)} + \bar{\mathcal{X}}_b^{(i)}\right) / 2$
  until the entire dataset is iterated through
end for

where $\bar{X}_n^{(i)}$ denotes the nth input sequence in the ith epoch, and $\hat{X}_n^{(i)}$ is its reconstruction generated by the model. The binary matrix $M_{x,n}$ represents the locations of missing values within the original sequence X_n, and the binary vector $m_{y,n}$ indicates the positions of missing values in the nth target, denoted as y_n. The predicted target values are represented by $\hat{y}_n$, and N is the number of training samples. Additionally, $\alpha^{(i)}$ and $\beta^{(i)}$ denote the adaptive coefficients associated with the reconstruction/prediction error of non-missing values and the imputation error of missing values, respectively. These coefficients adjust dynamically with the epoch number i, with *epochs* referring to the maximum number of epochs allowed. As i increases, $\alpha^{(i)}$ gradually transitions from 2 to 1, while $\beta^{(i)}$ changes from 0 to 1. $\|\cdot\|_F^2$ denotes the squared Frobenius norm that is the sum of the squares of all elements in the matrix, and $\|\cdot\|^2$ represents the squared Euclidean norm that is the sum of the squares of all individual elements in the vector.

Throughout the training process, the updating of model parameters and the imputation of missing values occur concurrently. The training procedure of the proposed model is as follows:

Initially, missing values are replaced with the mean of non-missing values from their respective timesteps. At each training epoch, the training dataset is divided into random batches of a chosen size B. Then, forward propagation is performed for each batch, where the data batch is fed into the model to generate both reconstructed sequences and future predictions. Following each forward propagation step, the mean of the input batch and its reconstruction is calculated using (4). Moreover, equation (5) is used to compute the loss for the current batch. Subsequently, the model's parameters are updated through backpropagation using a selected optimization algorithm. After that, any missing values in the input batch are imputed with the mean of the input data and its reconstruction obtained from the forward propagation. These steps are iterated for every batch in the training dataset during each epoch. The entire process is repeated for a specified number of epochs or until a predefined stopping criterion is satisfied. Algorithm 1 provides the pseudocode representation of the training process.

E. INFERENCE TIME COMPLEXITY ANALYSIS

The computational complexity of the proposed SI-DMSeq model at each inference step can be analyzed by considering its three main components:

1) ENCODER

The encoder consists of L stacked LSTM layers that process an input sequence of length T, where each element is a traffic vector of size Z (the number of zones). For each layer and timestep, the LSTM operations involve matrix multiplications with the hidden size h, which leads to a time complexity of $O\left(LTh^2 + ThZ\right)$ for the entire encoder.

2) RECONSTRUCTION DECODER

The reconstruction decoder is also composed of L stacked LSTM layers and reconstructs the input sequence of length T. At each decoding step, it involves an attention mechanism which adds a quadratic term with respect to the sequence length and hidden size. The total time complexity for this component is $O\left(LTh^2 + ThZ + T^2h^2\right)$.

3) PREDICTION DECODER

The prediction decoder, similarly, consists of L LSTM layers and predicts the traffic state for the next timestep. It also uses an attention mechanism, but since it generates only a single vector, its computational cost is lower. The complexity for this component is $O\left(Lh^2 + hZ + Th^2\right)$.

Considering all components and simplifying to dominant terms, the total inference time complexity of the SI-DMSeq model is $O\left(LTh^2 + ThZ + T^2h^2\right)$. This complexity scales linearly with the number of layers and zones, and quadratically with the sequence length and hidden size. Notably, the model's computational cost depends only on parameters fixed at deployment and is independent of the overall dataset size. Since the complexity grows only linearly with the number of zones (a key indicator of network size), the model is inherently scalable to large urban networks. This scalability enables SI-DMSeq to maintain low latency and robust performance in large-scale urban environments and supports near real-time applications in city-wide traffic systems.

F. TRAFFIC DISRUPTION DETECTION

1) NETWORK-LEVEL DISRUPTION DETECTION

When the SI-DMSeq model is trained on normal traffic data, it can effectively capture regular traffic patterns but is unable

to account for unusual patterns caused by an anomalous event. During the inference phase, the trained model can reliably reconstruct traffic states under normal conditions. However, when an extreme event disrupts the network, traffic patterns significantly change. Consequently, the model exhibits high reconstruction errors when confronted with these abnormal traffic states. By leveraging this limitation, we can detect network-wide disruptions by monitoring the reconstruction loss of the proposed model throughout the inference period.

Our monitoring system relies on Statistical Process Control (SPC) charts [31], which are powerful tools for detecting statistically significant deviations in a process. These charts plot a specific statistic over time and utilize upper and lower control limits (UCL and LCL) to identify abnormal variations. These limits are determined from historical data and represent the expected range of variation in the monitoring statistics. When the statistic exceeds these limits, it indicates the occurrence of a disruptive event.

To detect network-wide disruptions, we define our monitoring statistic as the Mean Square Error (MSE) of reconstruction across all the observed values in the input sequence. Specifically, at each timestep t, the monitoring statistic μ_t is calculated as follows:

$$\mu_t = \frac{\left\| \sim M_t \circ \left(\hat{X}_t - X_t \right) \right\|_F^2}{\text{sum} \left(\sim M_t \right)}, \tag{6}$$

where matrix $X_t \in \mathbb{R}^{Z \times T}$ denotes the input sequence at time t, which spans from timestep $(t - T + 1)$ to timestep t, with T representing the sequence length and Z denoting the number of traffic zones in the network. Each element of this matrix corresponds to the traffic value of a specific zone at a certain point in time. $\hat{X}_t$ denotes the reconstructed sequence obtained from the model, and the binary matrix M_t indicates the locations of missing values within the input sequence. The $\sim$ operator inverts the values of the binary matrix, converting each 0 to 1 and vice versa. The *sum* $(\cdot)$ function calculates the sum of all elements in its input matrix, and $\|\cdot\|_F^2$ denotes the squared Frobenius norm.

In our study, the statistical control chart is created based on the underlying distribution of our monitoring statistic. Specifically, we use historical data from a period of normal traffic conditions to empirically identify the distribution of μ_t. Based on this empirical distribution, we set the control limits of the chart using a preselected significance level. Any significant deviations from regular traffic patterns will manifest as the statistic μ_t falling outside the control limits, indicating the occurrence of a large-scale event within the network.

2) ZONE-LEVEL DISRUPTION DETECTION

During an extreme event, certain zones within the network may experience more anomalous traffic patterns. Various factors, including geographical location, population density, local infrastructure, and the nature of the event, contribute to varying degrees of disruptions across zones.

To identify the most disrupted zones in the network, we calculate anomaly scores for each zone over the inference period. These scores are obtained by imposing a sparsity-inducing penalty on the reconstruction loss of the model for each individual zone. Assuming the Mean Squared Error (MSE) of reconstruction for zone z at time t is denoted by e_{zt}, we calculate its anomaly score, denoted as s_{zt}, by applying a soft-thresholding operator [32] to the e_{zt} value:

$$s_{zt} = \text{sgn}\left(e_{zt}\right) \cdot max\left(|e_{zt}| - \lambda, 0\right), \tag{7}$$

where sgn $(\cdot)$ represents the sign function and λ denotes the threshold parameter. λ is adjusted to ensure that during non-event periods, the anomaly scores for most zones are zero. A higher score indicates more anomalous traffic patterns in a specific zone at a given time. This scoring approach allows usto identify which zones are more disrupted by the extreme event and prioritize our mitigation efforts accordingly.

IV. REAL WORLD APPLICATION AND PERFORMANCE EVALUATION

In this section, we present real-world case studies where we apply our proposed disruption detection method to the road network of Manhattan, New York City, using a publicly available traffic dataset [33]. Specifically, we evaluate its performance in identifying disruptions caused by Hurricane Sandy in 2012 and a Christmas blizzard in 2010. These events, along with several other disruptive events, both weather-related and non-weather-related, such as public celebrations, have been previously identified in the NYC dataset by [19] and validated using official event records. For this study, Hurricane Sandy and the Christmas blizzard were selected as representative examples of large-scale, weather-induced emergencies, which are particularly relevant for evaluating our method's ability to detect significant disruptions in urban traffic flow.

In this study, we focus on the Manhattan area due to its high traffic density, large-scale network, and the availability of high-quality traffic data that includes well-documented extreme events. These characteristics make it a compelling and demanding setting for evaluating disruption detection methods. While our evaluation is geographically limited to Manhattan, the proposed SI-DMSeq framework is designed to be generalizable and applicable to various urban areas.

For implementation and evaluation, we relied solely on general-purpose data analysis and deep learning libraries, such as pandas and PyTorch, without the use of specialized spatial analysis libraries. However, where appropriate, researchers can leverage open-source spatial analysis libraries, such as PySAL [34], to supplement data processing and enhance the analytical workflow. The source code developed for this study is publicly available on GitHub: https://github.com/minanouri/SI-DMSeq

A. DATASET DESCRIPTION AND PREPARATION

To evaluate our proposed method, we utilize an open-access and publicly available dataset derived from the historical

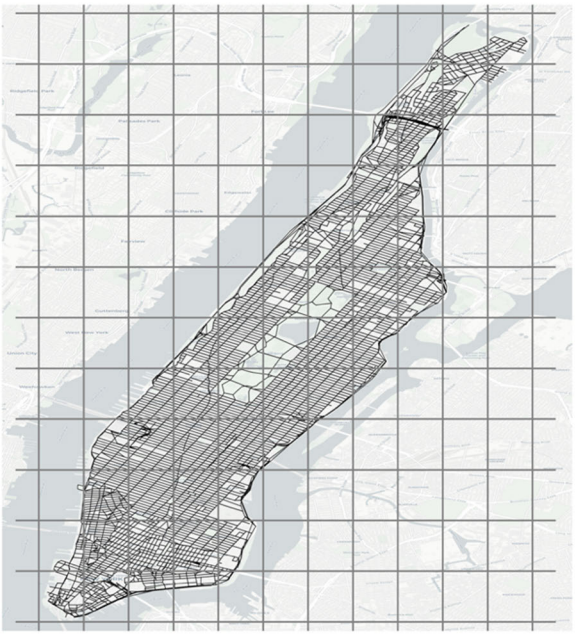


FIGURE 3. Partitioning of the Manhattan road network into zones.

trajectories of nearly 700 million taxi trips within New York City [33]. This dataset provides hourly average travel times for individual road segments throughout the city, spanning from January 2010 to December 2013. It also includes detailed information on the coordinates and length of each road segment.

To prepare the dataset for our model, we undertake a series of preprocessing steps outlined below:

Hurricane Sandy originated on October 22, 2012, and made landfall near Atlantic City, New Jersey, in the late evening of October 29, 2012. Accordingly, we select traffic data recorded prior to Monday, October 15, 2012, from the borough of Manhattan to construct and train the SI-DMSeq model. Additionally, as the proposed model necessitates training on normal traffic data, we filter out data recorded during periods of large-scale disruptions in the Manhattan road network. These disruptions, resulting from events such as blizzards, storms, and New Year celebrations, have been previously identified by [19]. Traffic data spanning from Monday, October 15, 2012 (two weeks before the hurricane), to Monday, November 5, 2012 (one week after the hurricane), is used to assess the model's performance in detecting disruptions caused by Hurricane Sandy.

Next, we group adjacent road segments into small zones. These zones are defined using a grid-based approach with equally-sized cells. A zone consists of road segments whose midpoints fall within the zone's boundaries defined by the grid system. As depicted in Fig. 3, the Manhattan road network is partitioned into forty-six zones.

Then, we obtain the hourly average pace of traffic within each zone. The average pace on a single road segment is defined as the average travel time divided by the road segment's length [35]. We calculate the average pace of a zone as the length-weighted average of the paces of all its constituent road segments. Specifically, the average pace of zone z at time t, denoted by x_{zt}, is determined as follows:

$$x_{zt} = \frac{\sum_{r \in S_z} l_r p_{rt}}{\sum_{r \in S_z} l_r}, \tag{8}$$

where S_z represents the set of all road segments within zone z, p_{rt} denotes the average pace on road segment r at timestep t, and l_r is the length of road segment r.

At each timestep t, the average pace values are arranged into a vector format, with each element corresponding to the average traffic pace within a specific zone of the network. This temporal sequence of vectors forms a multivariate time series.

To simulate a realistic deployment setting and ensure that model performance is validated on future unseen data, we split the time series chronologically and set aside the most recent 20% of the selected traffic data (i.e., data prior to October 15, 2012) as a validation set. This strategy prevents information leakage by preserving temporal order, which is critical for time series modeling, and reflects a real-world scenario where the model is trained on past data and applied to future observations. The choice of 20% follows standard practice in machine learning, where typically 20-25% of the available data is reserved for validation. This proportion provides a sufficient sample size for robust hyperparameter tuning while retaining enough data for effective model training. Additionally, we randomly remove 25% of the observations from the training data to assess the effectiveness of the proposed imputation strategy.

We use a sliding window approach to generate input-target pairs for training our SI-DMSeq model. Considering the daily patterns inherent in the traffic data, we select a fixed-size window with length $T = 24$. This window moves along the data one timestep at a time and creates a series of overlapping sequences, each spanning T timesteps. These sequences serve as the input for the model. The target for the reconstruction decoder is the original input sequence in reverse order, whereas for the prediction decoder, the target is the traffic vector for the subsequent timestep. Throughout the training process, the model learns to both reconstruct the input sequence and make one-step-ahead predictions by leveraging the patterns encoded within the input data.

B. MODEL CONFIGURATION

The proposed SI-DMSeq model is configured with specific settings as follows: The shared encoder employs two LSTM layers to extract important features from the input data. Both the reconstruction and prediction decoders utilize two LSTM layers followed by a Dense layer for their respective tasks. Every LSTM cell in the model and the Dense layers consist of 64 neurons. To further enhance generalization, a Dropout mechanism [36] with a probability of 0.2 is applied to the outputs of the lower LSTM layers within each component. The Adam optimizer [37], with a learning rate of 0.01, is employed to minimize the dynamic loss function presented

TABLE 1. Comparative analysis of imputation performance.

Percentage of missing data (ρ)	RMSE			
	Temporal Mean	RTRC	PARATUCK2	SI-DMSeq
10%	0.0744 (2e-3)	0.0344 (1e-3)	0.0360 (1e-3)	**0.0318** (9e-4)
20%	0.0747 (1e-3)	0.0357 (1e-3)	0.0361 (1e-3)	**0.0324** (1e-3)
30%	0.0748 (7e-4)	0.0365 (6e-4)	0.0366 (7e-4)	**0.0325** (5e-4)
40%	0.0750 (7e-4)	0.0374 (8e-4)	0.0366 (7e-4)	**0.0332** (5e-4)
50%	0.0754 (7e-4)	0.0388 (8e-4)	0.0377 (9e-4)	**0.0337** (6e-4)

in (5) across the training data. Training is conducted in mini-batches of size 64, and the model undergoes training for 20 epochs.

C. IMPUTATION PERFORMANCE EVALUATION

To evaluate the inference-phase imputation performance of our proposed model, we conduct a comparative analysis with three benchmark methods. The first benchmark is the naive Temporal Mean approach, where missing values are imputed using the average of non-missing values from the corresponding timestep. The second benchmark, the Robust Tensor Recovery and Completion (RTRC) method [22], [38], utilizes the Higher-order Robust Principal Component Analysis (PCA) technique to uncover the underlying low-rank structure inherent in typical traffic patterns. This structure is then used to regenerate the original tensor data and fill in missing values. The third benchmark is the PARATUCK2 imputation method [39], which employs a decomposition approach to capture spatial-temporal information of traffic data. This method creates a low-dimensional, clustered representation of traffic patterns and fills in missing values by reconstructing network traffic profiles from this representation.

Our evaluation is conducted under typical traffic conditions. Specifically, we utilize data collected between October 15 and October 29, 2012, and randomly mask $\rho = \{10, 20, 30, 40, 50\}$ percent of the available data as missing values. The remaining partial observations are then used as inputs for the models. To quantify the performance of inference-phase imputation, we use the Root Mean Squared Error (RMSE) metric, which is defined as follows:

$$\text{RMSE} = \sqrt{\frac{\sum_{\Omega} \left(\hat{x}_{zt} - x_{zt}\right)^2}{|\Omega|}}, \tag{9}$$

where Ω refers to the set of all masked values and $|\Omega|$ represents the total number of masked values. x_{zt} denotes the observed traffic value for zone z at time t, and $\hat{x}_{zt}$ represents its corresponding estimated value provided by the model.

Table 1 presents the mean and standard deviation of RMSE across 10 trials of inference-phase imputation for different percentages of missing data (ρ), using the proposed model and three benchmark approaches. The results demonstrate that our proposed approach outperforms the benchmarks at all specified levels of missing data. This underscores the effectiveness of our model's prediction component (i.e., Future Predictor) in imputing missing traffic data under typical conditions.

D. DISRUPTION DETECTION AT THE NETWORK LEVEL

Throughout the training process, the SI-DMSeq model adeptly learns to capture and recreate typical traffic patterns. However, it is unable to accommodate unusual patterns resulting from anomalous events. Consequently, while the trained Autoencoder effectively reconstructs traffic data under normal conditions, it tends to exhibit high reconstruction errors when faced with abnormal traffic states during extreme events. Using this limitation, we identify traffic disruptions at the network level by continuously monitoring the μ statistic (presented in (6)) over the inference period.

To achieve this, we develop an empirical control chart based on the underlying distribution of the μ statistic. Specifically, we utilize data spanning from October 15 to October 22, 2012 (referred to as Phase 1 in Fig. 4) to empirically identify the distribution of μ. This period is characterized by normal traffic condition, with no large-scale disruptions experienced by the network [19]. Using the Python library *distfit* [40], we determine that the Weibull distribution, with parameters shape=0.809, location=0.0006 and scale=0.0001, best fits the observed values of μ in Phase 1 (as illustrated in Fig. 5). Based on this distribution, we set the upper control limit (UCL) for μ at 0.0011 to achieve a 0.02 significance level. Data spanning from October 22 to November 5, 2012 (referred to as Phase 2 in Fig. 4) is subsequently used to monitor network traffic conditions and detect abnormalities caused by the hurricane event.

It is important to note that adjusting the significance level offers a flexible approach to detecting disruptions. Lower significance levels prioritize the detection of more severe disruptions while minimizing false alarms. Conversely, higher significance levels enhance sensitivity to smaller deviations but increase the likelihood of false alarms.

Fig. 4 displays the monitoring process. This figure reveals that the network encountered anomalous traffic patterns during October 30 to November 2, 2012, corresponding to the time when Hurricane Sandy affected New York City. Disruptions started around midnight on October 30, a few hours after

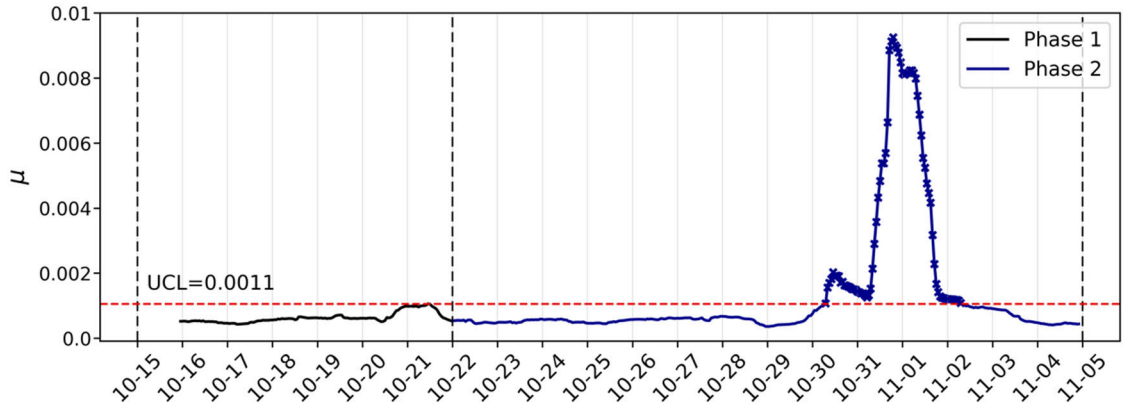


FIGURE 4. Hurricane Sandy case study -Monitoring process of the μstatistic. Traffic disruptions are detected around midnight on October 30, 2012, persisting until the morning of November 2, 2012.

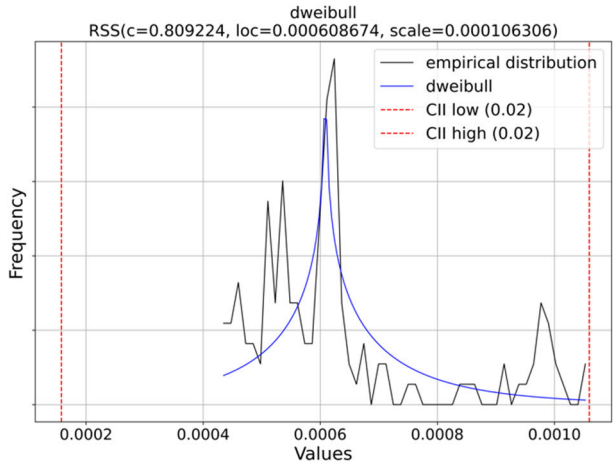


FIGURE 5. Hurricane Sandy case study - Best-fitted distribution to the observed values ofμ during Phase 1.

the hurricane made landfall near Atlantic City, New Jersey, and persisted until the morning of November 2, 2012. These observations align with earlier studies that have examined network-wide disruptions, such as those conducted by [19] and [20].

E. IDENTIFICATION OF THE MOST DISRUPTED ZONES

To assess the impact of Hurricane Sandy across different zones, we calculate anomaly scores for each zone at every timestep throughout the inference period. As detailed in Section III-F, these scores are calculated by applying a soft-thresholding operator to the reconstruction loss of the SI-DMSeq model for each specific zone. In this study, we set the threshold parameter λ to 0.0009. This threshold leads to 85% of the anomaly scores being zero during non-event periods.

Fig. 6 depicts the anomaly scores for all forty-six zones from October 22 to November 5, 2012. These scores quantify the extent of deviation from expected traffic patterns, with higher values indicating greater abnormalities in specific zones at given times. The impact of the hurricane event on the network is apparent in this figure.

To demonstrate the effectiveness of our proposed method in identifying disrupted zones, we employ a face validation approach. In this approach, we visually examine the pace time series of various zones with different anomaly scores. The findings confirm that zones with higher anomaly scores exhibit more significant deviations in their pace time series, while those with lower scores show minimal changes in pace values. For example, in Fig. 6, zones #8 and #25 receive high anomaly scores on October 31, 2012. This aligns with their pace time series in Fig. 7(a) and Fig. 7(b), respectively, which indicate severe congestion in these zones during that time. In contrast, the lower scores for zones #13 and #45 suggest that these zones were less affected by the hurricane event. Their pace time series in Fig. 7(c) and 7(d) confirm that they did not experience significant disruption during the event.

F. ADDITIONAL ANALYSIS WITH NYC BLIZZARD SCENARIO

To further evaluate the robustness of the proposed method, we assess its performance during a different but less extreme event, a powerful blizzard that hit New York City on December 26, 2010 [40]. Utilizing the New York City traffic dataset, the trained model is applied to data spanning from December 12, 2010 (two weeks prior to the event) to January 2, 2011 (one week after the event) in the Manhattan area.

Fig. 8 illustrates the monitoring of the μ statistic over the selected time period. The upper control limit in this chart is derived from the underlying distribution of μ values during Phase 1 (December 12 to December 19, 2010), which is calibrated to maintain a 0.02 significance level. The chart reveals that the network experienced anomalous traffic patterns during the blizzard event on December 26–27, 2010. Additionally, it highlights a continuation of these anomalies throughout the year-end period. These anomalies are attributed to both the delayed snow removal process [41] and the impact of the New Year event, which together contributed to significant shifts in traffic

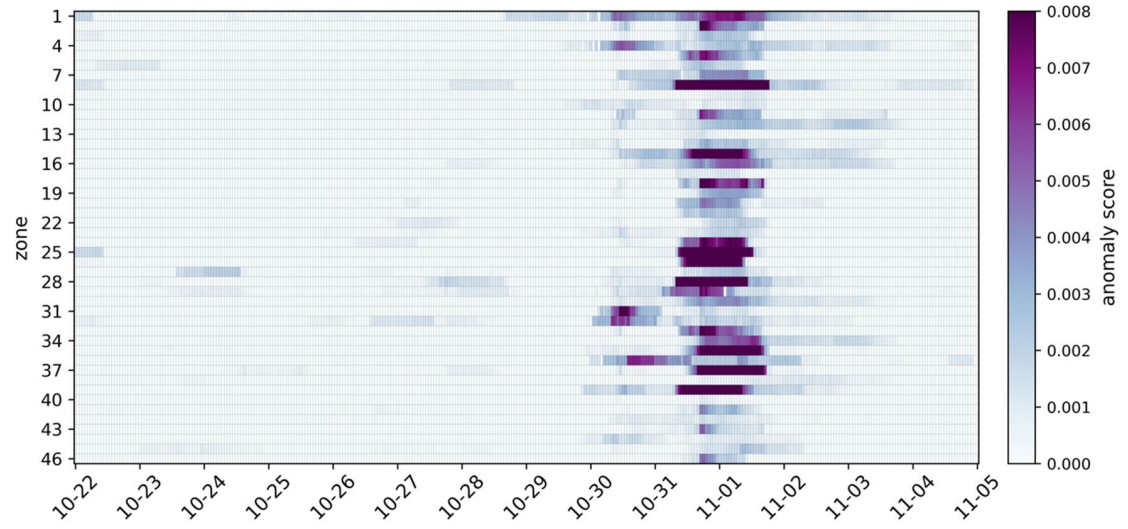


FIGURE 6. Hurricane Sandy case study -Anomaly scores for every zone during the inference period (from October 22 to November 5, 2012).

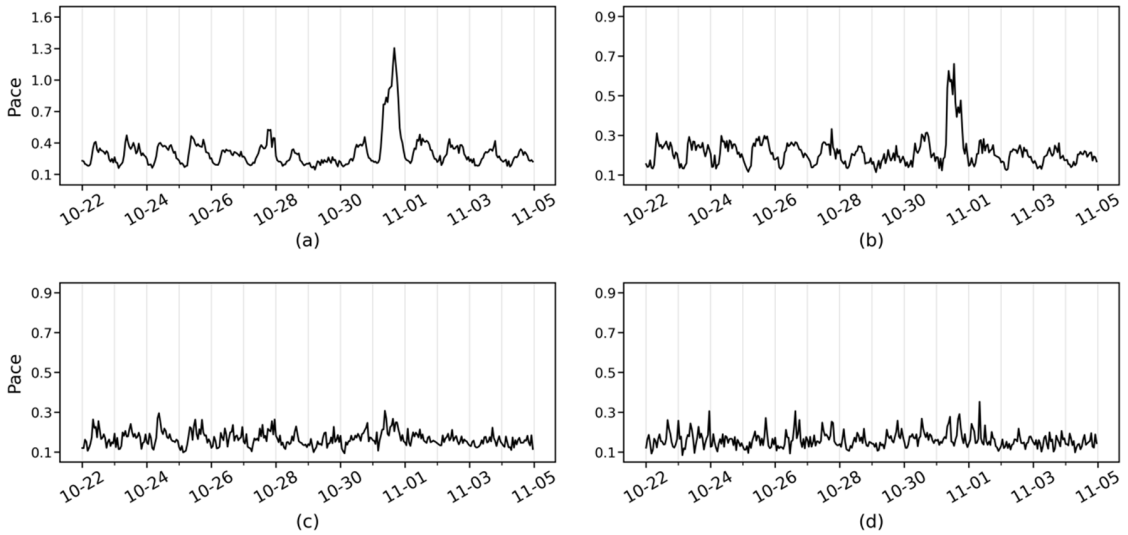


FIGURE 7. Hurricane Sandy case study -Pace time series of zones with high ((a) and (b)) and low ((c) and (d)) anomaly scores. (a) zone #8. (b) zone #25. (c) zone #13. (d) zone #45.

behavior. Similar patterns of network-wide disruptions during this period have also been identified in previous research ([19], [20]).

Figure 9 illustrates how traffic anomalies varied across the forty-six zones of the network from December 19, 2010, to January 2, 2011, based on anomaly scores. These scores represent the degree of deviation from expected traffic conditions, with higher values indicating more substantial disruptions. For example, the figure reveals that zones #8 and #16 experienced more intense anomalies during the year-end period compared to the Christmas blizzard. This observation aligns with the disruptions shown in their pace plots in Figures 10(a) and 10(b), particularly between December 28 and 31. In contrast, zone #10 recorded its highest anomaly scores during the blizzard on December 27, suggesting that it was more heavily affected by that event, as shown in Figure 10(c). Meanwhile, zone #21 displayed consistently low anomaly scores throughout the observed period, which indicates minimal impact from the events, as supported by its pace plot in Figure 10(d).

G. INFERENCE EFFICIENCY AND DEPLOYMENT CONSIDERATIONS

Although the original NYC dataset is large in scale, our proposed model is designed to process data efficiently by focusing on short, manageable sequences at each time step. Specifically, the model only processes sequences of length T, where each element is a traffic vector containing Z values, with Z representing the number of zones in the network. This design ensures that, at any given moment, the model is handling a fixed and relatively small amount of data, regardless of the overall dataset size. The computational

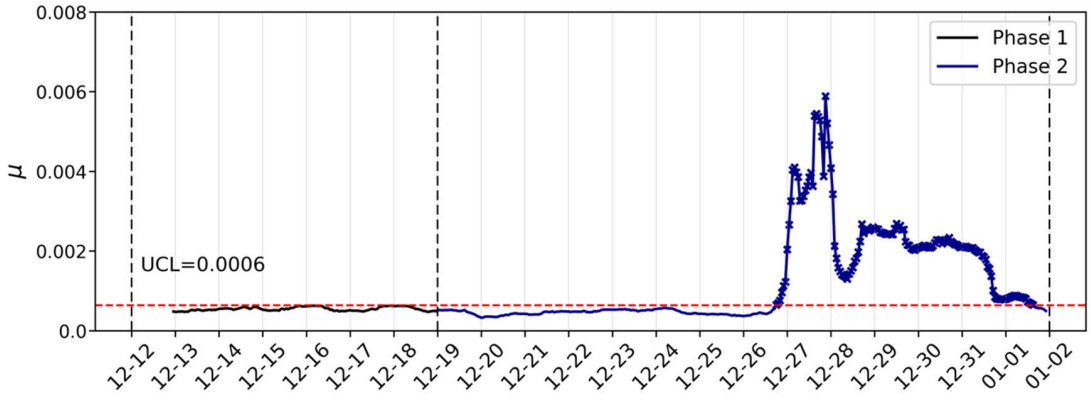


FIGURE 8. Christmas Blizzard case study -Monitoring of the μstatistic from December 12, 2010, to January 2, 2011. Traffic anomalies are detected during the blizzard event and the year-end period.

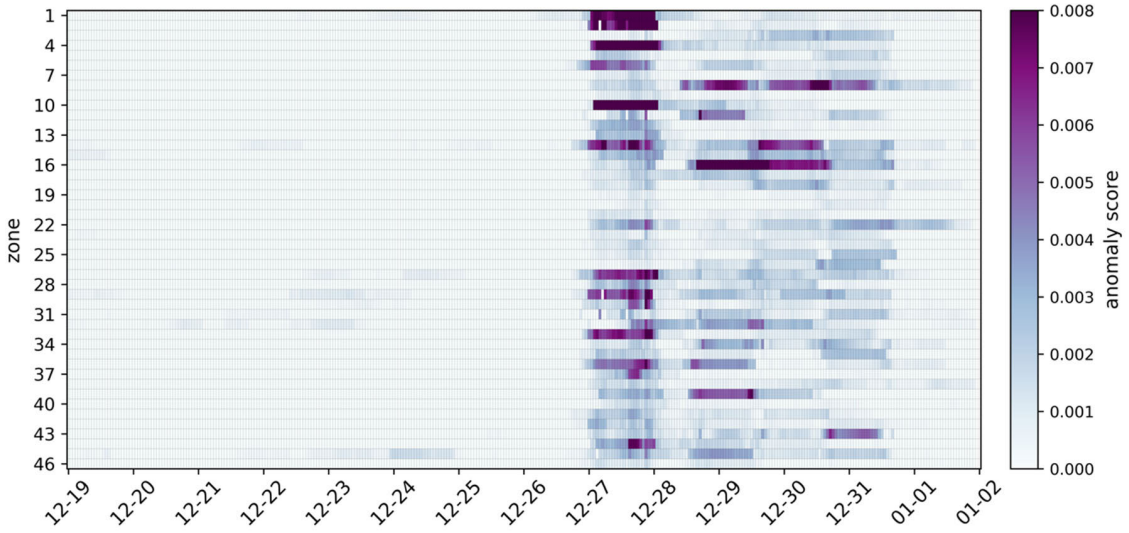


FIGURE 9. Christmas Blizzard case study - Anomaly scores for each traffic zone from December 19, 2010 to January 2, 2011.

complexity for each inference step scales linearly with the number of zones Z and quadratically with the sequence length T. This enables rapid analysis and timely detection of traffic disruptions, which makes the model suitable for near real-time applications.

Moreover, our data preprocessing pipeline is straightforward and computationally efficient. Core steps, including zone-level aggregation and sequence generation, are implemented using vectorized operations in standard data analysis libraries such as Pandas. These operations are highly optimized and support parallel execution, which enables the rapid preparation of input sequences. As a result, both the preprocessing and inference phases are streamlined to support timely and reliable disruption detection in large-scale urban environments.

To complement the theoretical complexity analysis, we evaluated the empirical inference time of the SI-DMSeq model using the NYC dataset on a MacBook Pro with a 2.3 GHz Quad-Core Intel Core i7 CPU and 32 GB RAM. For a sequence length of $T = 24$ and $Z = 46$ zones, the average inference time per sequence was approximately 55 milliseconds. This result confirms the model's suitability for near real-time deployment in real-world traffic monitoring applications.

While the dataset used in this study provides hourly average travel times, our proposed method is not inherently limited by this temporal resolution. It can seamlessly integrate with higher-frequency data sources, such as those providing travel times at 30-second intervals, if available. This flexibility enables potential deployment in near real-time traffic monitoring systems. By continuously processing streaming traffic data, the model can dynamically detect disruptions and provide timely support for informed decision-making.

H. LIMITATIONS AND FUTURE WORK

The current implementation of our proposed method employs a grid-based approach to define traffic zones, which has proven effective in Manhattan due to the uniformity of its road network. For cities with irregular or non-grid networks, the same framework could generalize by replacing the fixed grid with zones derived from network topology or spatial clustering. Future studies could explore data-driven methodologies for optimizing zone shapes and sizes based on local network characteristics. Developing adaptive zoning

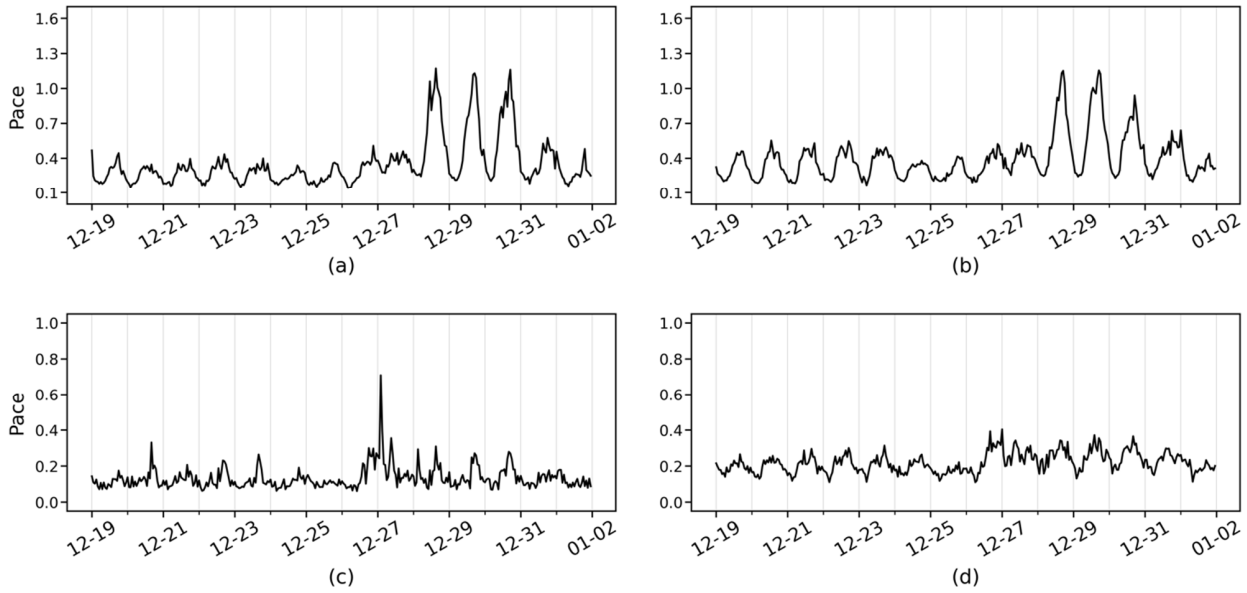


FIGURE 10. Christmas Blizzard case study -Pace time series of example zones with high ((a), (b) and (c)) and low ((d)) anomaly scores. (a) zone #8. (b) zone #16. (c) zone #10. (d) zone #21.

strategies for diverse urban environments could further enhance the model's applicability and performance.

While our model is designed to handle diverse traffic patterns and event scenarios, its current evaluation is limited to Manhattan and weather-related extreme events. To enhance the generalizability and robustness of the approach, future research should explore its applicability to cities with varying spatial layouts, traffic infrastructures, and congestion behaviors. Given the wide range of disruptive traffic events that affect cities globally, it is also important to expand the types of evaluated scenarios. Future research should include non-weather-related events, such as large-scale public gatherings and infrastructure failures, to fully assess the model's adaptability across diverse contexts. Furthermore, future work could explore classifying disruptive events based on the severity and spatial extent of their traffic impact. This would enhance the interpretability of detection results and support more targeted traffic management and response strategies.

In addition, future work could investigate the integration of more advanced architectures, such as Graph Neural Networks (GNNs) and transformer-based models, into the Autoencoder and Future Predictor components of the framework. These architectures have the potential to enhance spatiotemporal feature extraction and improve imputation performance, particularly in networks with complex topologies.

V. CONCLUSION

Recognizing the importance of reducing traffic congestion during extreme events, this study proposes an innovative Self-Imputing Deep Multitask Sequence (SI-DMSeq) model for dual-level traffic monitoring and disruption detection, which is specifically designed to work with incomplete datasets. The model is trained to perform two distinct tasks: (1) reconstructing sequences of traffic data and (2) generating one-step-ahead predictions. It adopts an iterative imputation strategy during training to handle missing traffic data. During inference, the reconstruction task enables disruption detection at both the network and local levels through an autoencoder-based approach, while the prediction task provides reliable imputation of missing values under typical traffic conditions.

We applied the proposed method to the road network of Manhattan, New York City, to evaluate its performance in detecting abnormal traffic patterns during Hurricane Sandy in 2012 and the Christmas blizzard in 2010. The results demonstrate the method's ability to accurately and simultaneously detect event-induced traffic disruptions at both the network and zone levels, while efficiently imputing missing data throughout the monitoring process.

Although Hurricane Sandy was well-predicted from a meteorological perspective, its impact on the transportation network, particularly at the road segment level, was a complex and dynamic spatiotemporal phenomenon that could not be easily anticipated. Additionally, the blizzard that affected New York City on December 26, 2010, resulted in unforeseeable traffic disruptions influenced by a variety of factors, such as snow removal delays and holiday-related activities. Our proposed method offers a data-driven approach for near-real-time traffic monitoring and disruption detection. By continuously analyzing traffic conditions, our method provides valuable insights that can support emergency response efforts and improve traffic management before, during, and after large-scale event.

ACKNOWLEDGMENT

Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

[1] J. E. Abanto, C. C. Reyes, J. A. Malinao, and H. N. Adorna, "Traffic incident detection and modelling using quantum frequency algorithm and AutoRegressive integrated moving average models," in *Proc. IISA*, Jul. 2013, pp. 1–6.

[2] H. Li, S.-L. Li, H. Zhu, X. Zhao, and X. Zhang, "Automated detection algorithm for traffic incident in urban expressway based on lengthways time series," in *Proc. 8th Int. Conf. Green Intell. Transp. Syst. Saf.*, 2018, pp. 625–633.

[3] F. Harrou, A. Zeroual, and Y. Sun, "Traffic congestion monitoring using an improved kNN strategy," *Measurement*, vol. 156, May 2020, Art. no. 107534.

[4] J. Evans, B. Waterson, and A. Hamilton, "A random forest incident detection algorithm that incorporates contexts," *Int. J. Intell. Transp. Syst. Res.*, vol. 18, no. 2, pp. 230–242, May 2020.

[5] N. Dogru and A. Subasi, "Traffic accident detection using random forest classifier," in *Proc. 15th Learn. Technol. Conf. (L&T)*, Feb. 2018, pp. 40–45.

[6] Y. Sun, T. Mallick, P. Balaprakash, and J. Macfarlane, "A data-centric weak supervised learning for highway traffic incident detection," *Accident Anal. Prevention*, vol. 176, Oct. 2022, Art. no. 106779.

[7] B. Yao, P. Hu, M. Zhang, and M. Jin, "A support vector machine with the Tabu search algorithm for freeway incident detection," *Int. J. Appl. Math. Comput. Sci.*, vol. 24, no. 2, pp. 397–404, Jun. 2014.

[8] J. Xiao, "SVM and KNN ensemble learning for traffic incident detection," *Phys. A, Stat. Mech. Appl.*, vol. 517, pp. 29–35, Mar. 2019.

[9] A. B. Parsa, H. Taghipour, S. Derrible, and A. K. Mohammadian, "Real-time accident detection: Coping with imbalanced data," *Accident Anal. Prevention*, vol. 129, pp. 202–210, Aug. 2019.

[10] D. Srinivasan, X. Jin, and R. L. Cheu, "Adaptive neural network models for automatic incident detection on freeways," *Neurocomputing*, vol. 64, pp. 473–496, Mar. 2005.

[11] L. Zhu, F. Guo, R. Krishnan, and J. W. Polak, "A deep learning approach for traffic incident detection in urban networks," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Nov. 2018, pp. 1011–1016.

[12] Q. Shang, L. Feng, and S. Gao, "A hybrid method for traffic incident detection using random forest-recursive feature elimination and long short-term memory network with Bayesian optimization algorithm," *IEEE Access*, vol. 9, pp. 1219–1232, 2021.

[13] L. Li, Y. Lin, B. Du, F. Yang, and B. Ran, "Real-time traffic incident detection based on a hybrid deep learning model," *Transportmetrica A, Transp. Sci.*, vol. 18, no. 1, pp. 78–98, Mar. 2022.

[14] Y. Lin, L. Li, H. Jing, B. Ran, and D. Sun, "Automated traffic incident detection with a smaller dataset based on generative adversarial networks," *Accident Anal. Prevention*, vol. 144, Sep. 2020, Art. no. 105628.

[15] S. Yang, K. Kalpakis, and A. Biem, "Detecting road traffic events by coupling multiple timeseries with a nonparametric Bayesian method," *IEEE Trans. Intell. Transp. Syst.*, vol. 15, no. 5, pp. 1936–1946, Oct. 2014.

[16] L. Tišljarić, S. Fernandes, T. Carić, and J. Gama, "Spatiotemporal traffic anomaly detection on urban road network using tensor decomposition method," in *Proc. Int. Conf. Discovery Sci.*, Oct. 2020, pp. 674–688.

[17] Q. Li, H. Tan, Z. Jiang, Y. Wu, and L. Ye, "Nonrecurrent traffic congestion detection with a coupled scalable Bayesian robust tensor factorization model," *Neurocomputing*, vol. 430, pp. 138–149, Mar. 2021.

[18] S. E. Sofuoglu and S. Aviyente, "GLOSS: Tensor-based anomaly detection in spatiotemporal urban traffic data," *Signal Process.*, vol. 192, Mar. 2022, Art. no. 108370.

[19] B. Donovan and D. B. Work, "Empirically quantifying city-scale transportation system resilience to extreme events," *Transp. Res. C, Emerg. Technol.*, vol. 79, pp. 333–346, Jun. 2017.

[20] M. Ilbeigi, "Statistical process control for analyzing resilience of transportation networks," *Int. J. Disaster Risk Reduction*, vol. 33, pp. 155–161, Feb. 2019.

[21] Y. Hu, A. Qu, and D. Work, "Detecting extreme traffic events via a context augmented graph autoencoder," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 6, pp. 1–23, Sep. 2022.

[22] Y. Hu and D. B. Work, "Robust tensor recovery with fiber outliers for traffic events," *ACM Trans. Knowl. Discovery Data*, vol. 15, no. 1, pp. 1–27, Dec. 2020.

[23] E. Siri, S. Siri, and S. Sacone, "Network performance evaluation under disruptive events through a progressive traffic assignment model," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 15017–15022, Jan. 2020.

[24] T. Liu and H. Meidani, "End-to-end heterogeneous graph neural networks for traffic assignment," *Transp. Res. C, Emerg. Technol.*, vol. 165, Aug. 2024, Art. no. 104695.

[25] T. Liu and H. Meidani, "Heterogeneous graph sequence neural networks for dynamic traffic assignment," 2024, *arXiv:2408.04131*.

[26] G. Fusco, C. Colombaroni, L. Comelli, and N. Isaenko, "Short-term traffic predictions on large urban traffic networks: Applications of network-based machine learning models and dynamic traffic assignment models," in *Proc. Int. Conf. Models Technol. Intell. Transp. Syst. (MT-ITS)*, Jun. 2015, pp. 93–101.

[27] M.-T. Luong, Q. V. Le, I. Sutskever, O. Vinyals, and L. Kaiser, "Multi-task sequence to sequence learning," 2015, *arXiv:1511.06114*.

[28] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[29] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.

[30] Z. Pan, Y. Wang, K. Wang, H. Chen, C. Yang, and W. Gui, "Imputation of missing values in time series using an adaptive-learned median-filled deep autoencoder," *IEEE Trans. Cybern.*, vol. 53, no. 2, pp. 695–706, Feb. 2023.

[31] D. C. Montgomery, *Introduction to Statistical Quality Control*. Hoboken, NJ, USA: Wiley, 2009.

[32] T. Hastie, R. Tibshirani, and M. Wainwright, *Statistical Learning With Sparsity: The Lasso and Generalizations*. Boca Raton, FL, USA: CRC Press, 2015.

[33] B. Donovan and D. B. Work. (Jun. 2015). *Open Data: 2010–2013 New York City Traffic Estimates*. University of Illinois. [Online]. Available: https://uofi.app.box.com/v/NYC-traffic-estimates

[34] S. J. Rey and L. Anselin, "PySAL: A Python library of spatial analytical methods," in *Handbook of Applied Spatial Analysis: Software Tools, Methods and Applications*. Berlin, Germany: Springer, 2009, pp. 175–193.

[35] C. F. Daganzo, *Fundamentals of Transportation and Traffic Operations*. Pergamon, Oxford, 1997.

[36] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

[38] D. Goldfarb and Z. Qin, "Robust low-rank tensor recovery: Models and algorithms," *SIAM J. Matrix Anal. Appl.*, vol. 35, no. 1, pp. 225–253, Jan. 2014.

[39] M. Nouri, M. Reisi-Gahrooei, and M. Ilbeigi, "A method for granular traffic data imputation based on PARATUCK2," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 2676, no. 10, pp. 220–230, Oct. 2022.

[40] E. Taskesen. (Oct. 2020). *Distfit is a Python Library for Probability Density Fitting*. Python Library. [Online]. Available: https://erdogant.github.io/distfit

[41] Wikipedia. (2010). *December 2010 North American Blizzard*. Accessed: Mar. 15, 2025. [Online]. Available: https://en.wikipedia.org/wiki/December_2010_North_American_blizzard



MINA NOURI received the B.S. and M.S. degrees in civil engineering from the University of Tehran, Iran, and the Ph.D. degree in civil engineering from Stevens Institute of Technology, Hoboken, NJ, USA. Her research interests include machine learning, big data analytics, and anomaly detection, with applications to real-world systems.



MOHAMMAD ILBEIGI received the B.Sc. degree in civil engineering from the Sharif University of Technology, the M.Sc. degree in civil engineering from the University of Tehran, Iran, and the M.Sc. degree in computational science and engineering and the Ph.D. degree in construction management from Georgia Institute of Technology, Atlanta, GA, USA. He is currently an Assistant Professor with the Department of Civil, Environmental, and Ocean Engineering, Stevens Institute of Technology in Hoboken, NJ, USA. His research interest includes resilient and adaptive infrastructure development.